

IP1 Feature Map

XR Renovation Previewer · DECO7230 · Kaike Nehme · Fri 28 Aug 2026 · All paths under ip1/RenovationPreviewer/Assets/ · EditMode tests 61/61 (Scripts/Tests/) · PlayMode 8/8 (Scripts/PlayModeTests/SceneWiringTests.cs)

Architecture rule (concept §8): surfaces never know samples or menus exist. Surface is a state machine; HoldUpPreviewer is the *sole* coupling point between a held sample and a surface; the menu commits through the same Surface preview/commit API the hold-up loop uses. One write path — that's why menu and hold-up can be compared fairly (assumption A3).

The core loop — mark · pull · hold · tune · commit

Feature	Implementation	How it works
Mark (keep/change)	Scripts/Runtime/MarkTool.cs · Surface.cs	Brushing a controller across a surface while holding grip toggles Keep/Change. Grip gate + sweep-speed threshold stop accidental toggles (open Q3); per-surface debounce stops double fires.
Pull (sample source)	Scripts/Runtime/SamplePuller.cs · Sample.cs	Touch a Keep surface + trigger → spawns a sample already held by that controller. Pulling from a Change surface is refused — only kept things are sources; that is the concept (§9).
Hold-up preview	Scripts/Runtime/HoldUpPreviewer.cs	While held: nearest Change surface in range previews the sample colour; leaving range always reverts (a preview must never stick, §9). Sole coupling point between Sample and Surface.
Tune (twist)	Scripts/Runtime/HarmonyTuner.cs · HarmonyPalette.cs	Controller roll (relative to grab pose) indexes a 7-option palette generated from the pulled colour, ordered safe→bold: analogous pair, tonal pale/deep, triadic pair, complementary.
Commit / discard	HoldUpPreviewer.cs · Surface.cs	Release in range commits the previewed look to the surface; release mid-air discards the sample. Surface stores committed vs previewed colour AND material separately.

Diegetic controls — no floating menus

Feature	Implementation	How it works
Light (lamp)	Scripts/Runtime/LampController.cs	Touching the lamp cycles Warm → Cool → Daylight, relighting the room. Three states, not on/off: "does this green survive a warm bulb at 9pm?" (§7).
Compare (schemes)	Scripts/Runtime/SchemeCycler.cs · SchemeManager.cs	Primary button saves the current scheme (max 3, colour+material snapshot per surface); secondary cycles saved schemes. Physical controller buttons — the no-menus rule holds.

Controller menu (added 25 Aug for the A3 comparison)

Feature	Implementation	How it works
Open / target	Scripts/Runtime/MenuSelectRelay.cs · MenuTarget.cs · RayUtil.cs	Right trigger fires a physics ray → anything with MenuTarget opens the menu for it. Ray skips the rig's own colliders and never treats the menu panel as a wall (tag MenuPanel). Trigger on nothing closes.
Panel v2	Scripts/Runtime/ControllerMenu.cs (361 ln) · SwatchButton.cs · MenuTab.cs · Uikit.cs	World-space canvas above the left hand, facing the eyes: header, segmented tabs (Paint / Material / Add furniture), 4x2 paged chips with thumbnails, ✓ on the committed option. Hover previews via the same Surface API; click commits; moving off reverts. All sprites procedural (Uikit) — zero art assets.
Catalogue	Scripts/Runtime/Catalogue.cs · Editor/CatalogueImporter.cs (376 ln)	One-shot idempotent importer: Poly Haven CC0 glTF furniture, ambientCG CC0 PBR materials, Dulux public colour data → URP materials + prefabs + Catalogue.asset. Bundled, no runtime downloads.

Furniture & locomotion

Feature	Implementation	How it works
Move (grip-grab)	Scripts/Runtime/FurnitureSlot.cs	Every furniture piece is a slot: stable GameObject holds pose, box collider, XRGrabInteractable (kinematic, no throw); the

Feature	Implementation	How it works
		visual is a swappable child. Grip grabs, drag with the ray; release snaps flat to floor, clamped inside the room. Discoverability = data-sheet row 16.
Swap / add / remove	FurnitureSlot.Swap() / .Spawn() / .Remove()	Menu's furniture tab spawns a new slot at the aimed floor point or swaps the targeted slot's visual; collider auto-fits renderer bounds; sofa is a slot too (kept = also a sample source).
Teleport	Editor/SceneBuilder.cs · RoomSpec.cs	Floor is a TeleportationArea on interaction layer 31 (reserved in RoomSpec.TeleportLayer) so teleport rays and select rays never collide. Point at floor + trigger.
Aim feedback	Scripts/Runtime/RayFeedback.cs · RayUtil.cs	4 cm reticle oriented to the hit normal (safe up-vector — the Invalid-AABB fix) + emission glow on whatever the ray points at. Pure per-frame state machine.

Scene, rig & build infrastructure

Piece	Implementation	Notes
Scene generation	Editor/SceneBuilder.cs (310 ln)	Room.unity is <i>generated, never hand-edited</i> : 5×4×2.7 m room (RoomSpec), kept timber floor + sofa, 4 changeable walls + ceiling + trim + door, lamp, rig start pose facing the sofa. Re-runnable from scratch.
XR rig / input	XRI 3.1.2 Starter Assets rig prefab + code-created InputActions	Tool inputs bind to <XRController>{Left Right}Hand paths directly, not the Starter action asset. Controllers only — hand tracking stripped (hurdle-risk decision). NearFar interactors do grab + UI.
Project config	Editor/ProjectConfigurator.cs	One-shot URP + OpenXR (Meta Quest feature) + Android IL2CPP/ARM64 setup.
Build	Editor/BuildScript.cs	BuildAndroid → Builds/ip1.apk, headless via -batchmode - executeMethod.
Simulator QoL	Scripts/Runtime/SimulatorCursorLock.cs	Editor-only (#if UNITY_EDITOR): auto-locks the mouse in play mode for XR Device Simulator runs. Compiled out of the Quest build.
Tests	Scripts/Tests/ · Scripts/PlayModeTests/	61 EditMode unit tests (palette maths, tuner, surfaces, schemes, menu, ray utils, furniture) + 8 PlayMode scene-wiring tests (rig, layers, teleport, menu bindings) — all green at submission.

Input cheat-sheet: trigger = pull sample / select / commit chip / teleport · grip = mark sweep / grab furniture · primary = save scheme · secondary = cycle schemes / close menu · touch lamp = light state. Simulator keys: Tab cycle device, left-click = trigger, G = grip, \ = cursor lock.